МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

Кафедра «Интеллектуальные информационные технологии»

ЛАБОРАТОРНАЯ РАБОТА №3

по дисциплине «Аппаратное и программное обеспечение сетей» за 4 семестр

Тема: «анализ сетевого трафика и протоколов на базе Wireshark»

Вариант 5

Выполнил:
студент 3 курса
группы ИИ-28
(1 подгруппа)
Деркач Е. В.
Проверил:
Кулик А.Д.

Брест 2026

Цель работы: изучить возможности и приёмы работы со сниффером Wireshark, освоить типы фильтрации трафика, правила построения фильтров и приёмы статистической обработки сетевого трафика, а также на практике проанализировать работу протоколов Ethernet, IP, TCP, UDP, ICMP и ARP.

Ход работы

Задания:

1. Изучить краткие теоретические сведения по возможностям и приёмам работы с программой Wireshark (файл netWS.pdf).
2. Изучить типы фильтрации трафика, правила построения фильтров и приёмы статистической обработки сетевого трафика в Wireshark (материал приведён ниже).
3. Запустив Wireshark на захват, выполнить загрузку доступной в лабораторных условиях страницы (bstu.by, iit.bstu.by или др.). Остановить и сохранить захват. Для захваченных пакетов определить статистические данные:

- процентное соотношение трафика разных протоколов в сети;
- среднюю скорость кадров/сек;
- среднюю скорость байт/сек;
- минимальный, максимальный и средний размеры пакета;
- степень использования полосы пропускания канала (загрузку сети).

4. Отфильтровать в захвате IP-пакеты. Определить статистические данные:

- процентное соотношение трафика разных протоколов стека TCP/IP в сети;
- средний, минимальный, максимальный размеры пакета.

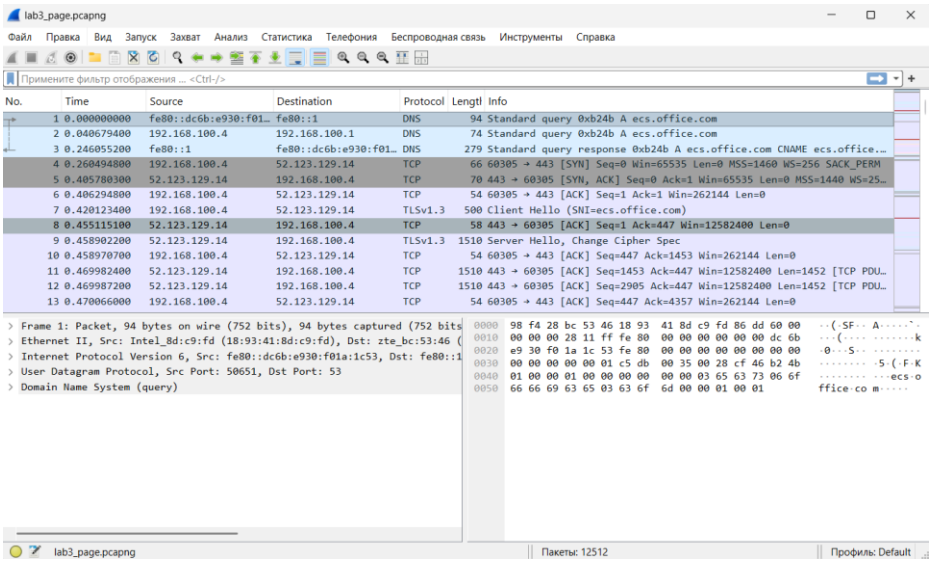
На примере любого IP-пакета указать структуры протоколов Ethernet и IP. Отметить поля заголовков, описать их и интерпретировать их значения.

5. Запустив Wireshark на захват, выполнить команду ping для IP-адреса соседней рабочей станции в лаборатории (предварительно определив её адрес с помощью ipconfig). Сохранить результат. Сформировав нужный фильтр, отфильтровать пакеты, относящиеся к выполнению команды ping. На базе полученных пакетов и значений их полей интерпретировать результат работы утилиты ping. Описать все протоколы, используемые утилитой. Составить диаграмму взаимодействия машин при работе утилиты ping. Примечание: данная утилита использует протокол ICMP (RFC 792 и RFC 960).
6. Запустив Wireshark на захват, выполнить разработанный в предыдущей лабораторной работе TCP-сервер и TCP-клиент. Сохранить результат. Сформировав нужный фильтр, отфильтровать пакеты данного сеанса. На примере любого TCP-сегмента указать структуру протокола TCP. Отметить поля заголовка, описать их и интерпретировать их значения. Составить диаграмму взаимодействия TCP-сервера и TCP-клиента.
7. Запустив Wireshark на захват, выполнить разработанный в предыдущей лабораторной работе UDP-сервер и UDP-клиент. Сохранить результат. Сформировав нужный фильтр, отфильтровать пакеты данного сеанса. На примере любого UDP-сегмента указать структуру протокола UDP. Отметить поля заголовка, описать их и интерпретировать их значения.
8. Выполнить анализ ARP-протокола по примеру из методических указаний.

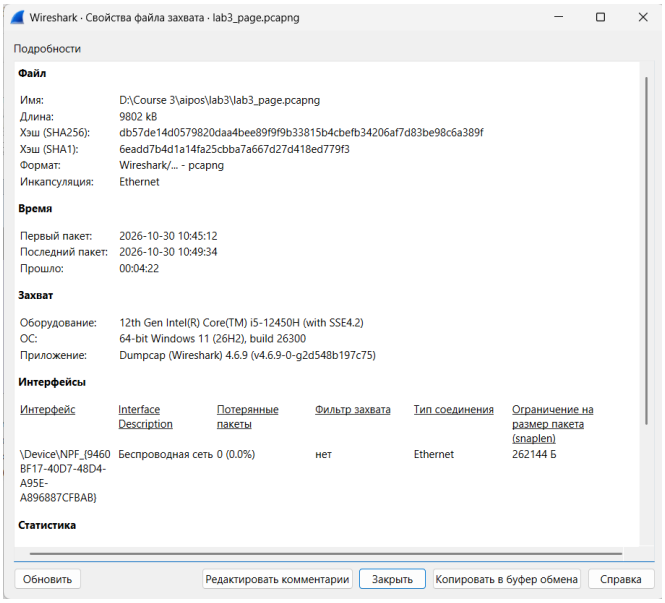
Решения задач:

3. Захват и статистический анализ трафика при загрузке страницы bstu.by

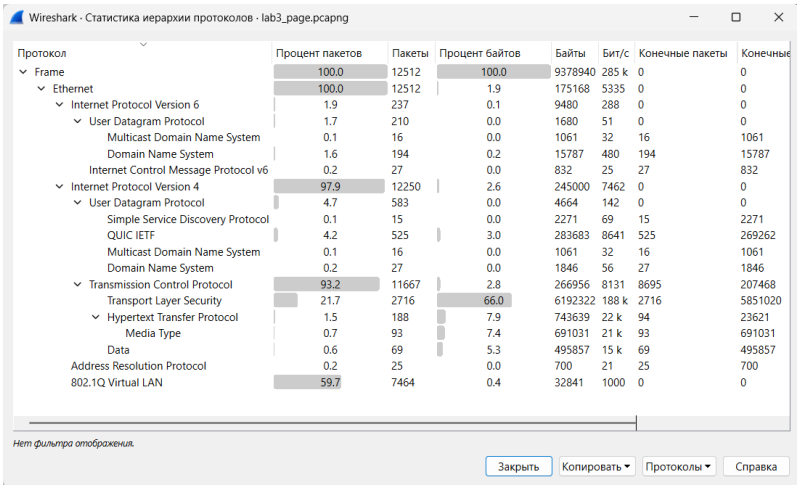
Захват прошел успешно: компьютер (IP 192.168.100.4) сначала делает DNS-запрос (чтобы узнать IP сайта), а затем устанавливает TCP-соединение (пакеты SYN, SYN, ACK, ACK). Файл сохранен (lab3_page.pcapng).



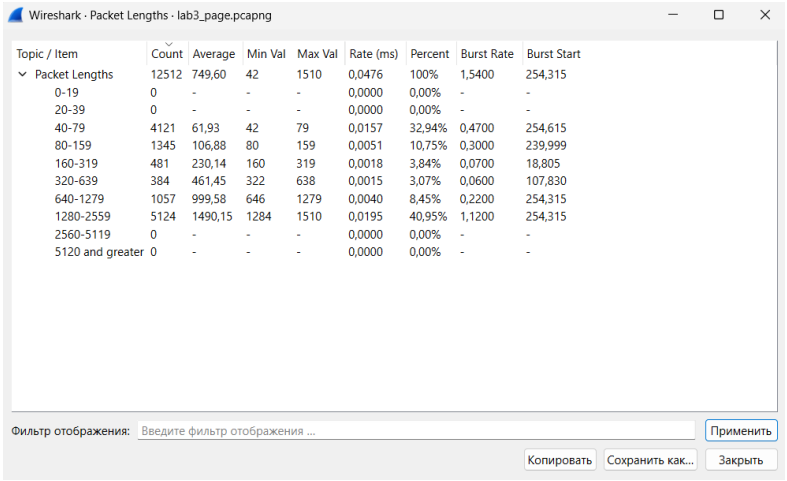
Общие сведения о захваченном трафике (время, размер файла, интерфейс):
Интерфейс: Беспроводная сеть. Время захвата: 4 мин 22 сек (262 с). Всего пакетов: 12 512. Общий объем: 9 378 940 байт.



Иерархия протоколов и процентное соотношение трафика:
Процентное соотношение трафика: IPv4 — 97,9%, IPv6 — 1,9%, ARP — 0,2%. Внутри IPv4: TCP — 93,2%, UDP — 4,7%. На прикладном уровне: TLS — 21,7%, HTTP — 1,5%.

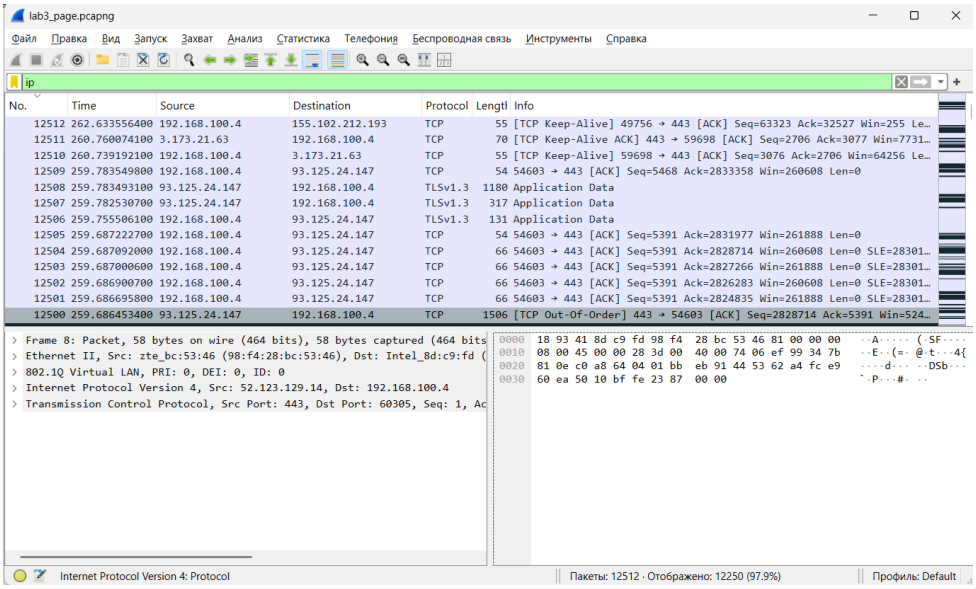


Распределение пакетов по длине (минимальный, максимальный и средний размер): Размеры пакетов: минимальный — 42 байта, максимальный — 1510 байт, средний — 749,60 байт. Средняя скорость: 47,6 пакетов/с. Средняя скорость байт/сек: 35 797 байт/с (9 378 940 / 262). Загрузка полосы пропускания: 0,28 Мбит/с (при канале 100 Мбит/с — 0,28%).

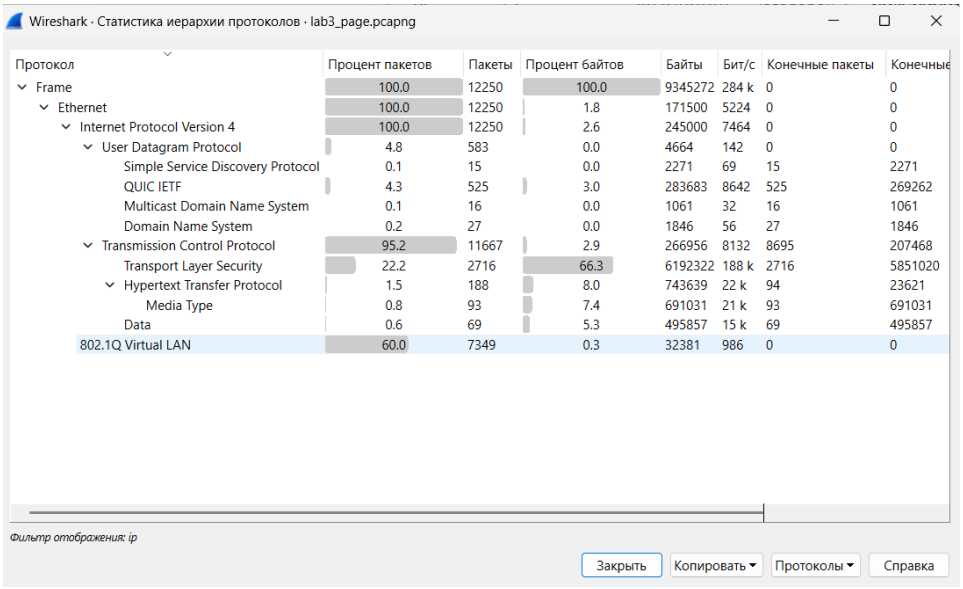


4. Фильтрация IP-пакетов и анализ структур протоколов Ethernet и IP

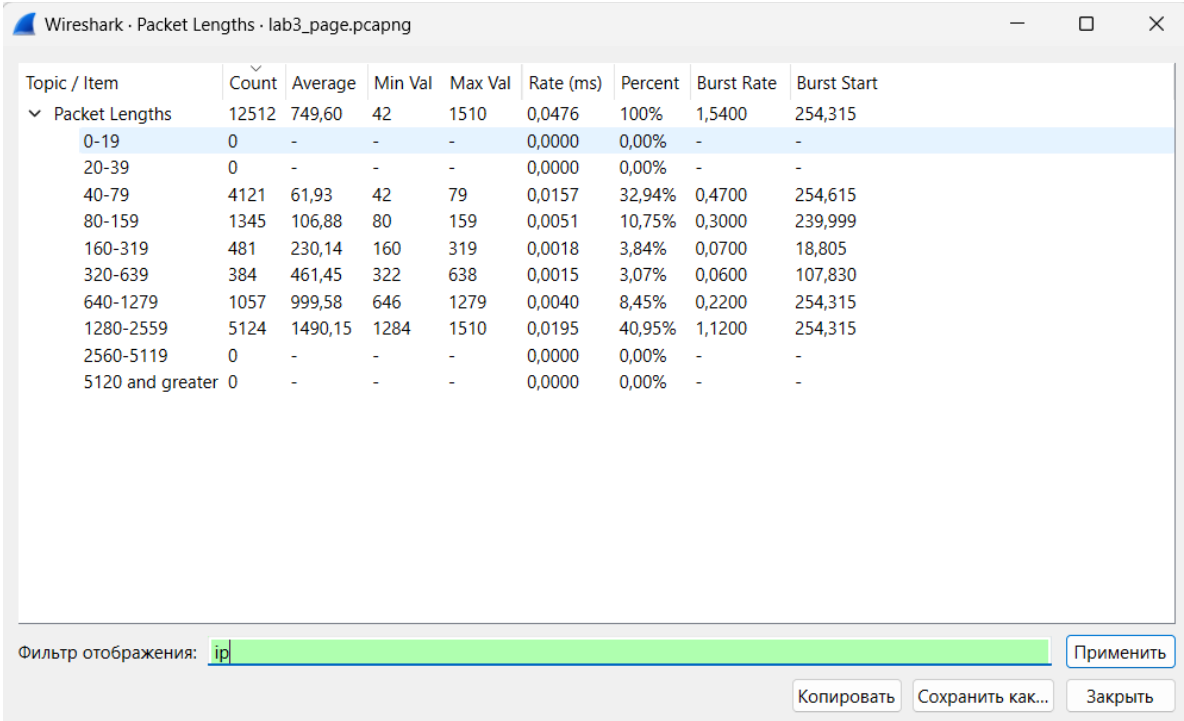
Для выделения сетевого трафика был применен фильтр отображения ip. В результате фильтрации в захвате осталось 12 250 IP-пакетов (из них 100% — IPv4).



Иерархия протоколов и процентное соотношение трафика:
Процентное соотношение трафика стека TCP/IP: TCP — 95,2%, UDP — 4,8%.



Распределение пакетов по длине (минимальный, максимальный и средний размер):
Минимальный размер пакета: 42 байта;
Максимальный размер пакета: 1510 байт;
Средний размер пакета: 749,60 байт.



Структура протокола Ethernet II:
Заголовок канального уровня:

- Destination (MAC-адрес получателя): zte_bc:53:46 (98:f4:28:bc:53:46) — физический адрес сетевой карты получателя;
- Source (MAC-адрес отправителя): Intel_8d:c9:fd (18:93:41:8d:c9:fd) — физический адрес отправителя;
- Type (Тип): IPv4 (0x0800) — указывает, что внутри кадра находится пакет IPv4.

Структура протокола IP (IPv4):

- Version (Версия): 4 — используется протокол IPv4;

- Header Length (Длина заголовка): 20 байт (5) — стандартная длина заголовка;
- Total Length (Общая длина): 41 байт — длина самого IP-пакета;
- Identification (Идентификатор): 0x5262 (21090);
- Flags (Флаги): 0x2, Don't fragment — пакет не подлежит фрагментации;
- Time to Live (TTL): 128 — время жизни пакета;
- Protocol (Протокол): TCP (6) — указывает, что внутри IP-пакета находится сегмент TCP;
- Header Checksum (Контрольная сумма): 0x0000;
- Source Address (IP-адрес отправителя): 192.168.100.4;
- Destination Address (IP-адрес получателя): 155.102.212.193.

5. Анализ работы утилиты ping и протокола ICMP

Для анализа работы утилиты ping был запущен захват трафика в Wireshark. С помощью команды ipconfig был определен основной шлюз лабораторной сети — 192.168.100.1 (IP-адрес нашей машины 192.168.100.4).

```
Адаптер беспроводной локальной сети Беспроводная сеть:

   DNS-суффикс подключения . . . . . :
   Локальный IPv6-адрес канала . . . : fe80::dc6b:e930:f01a:1c53%13
   IPv4-адрес. . . . . . . . . . . . : 192.168.100.4
   Маска подсети . . . . . . . . . . : 255.255.255.0
   Основной шлюз. . . . . . . . . : fe80::1%13
                                     192.168.100.1
```

В командной строке была выполнена команда ping 192.168.100.1. После получения 4 ответов захват был остановлен и сохранен.

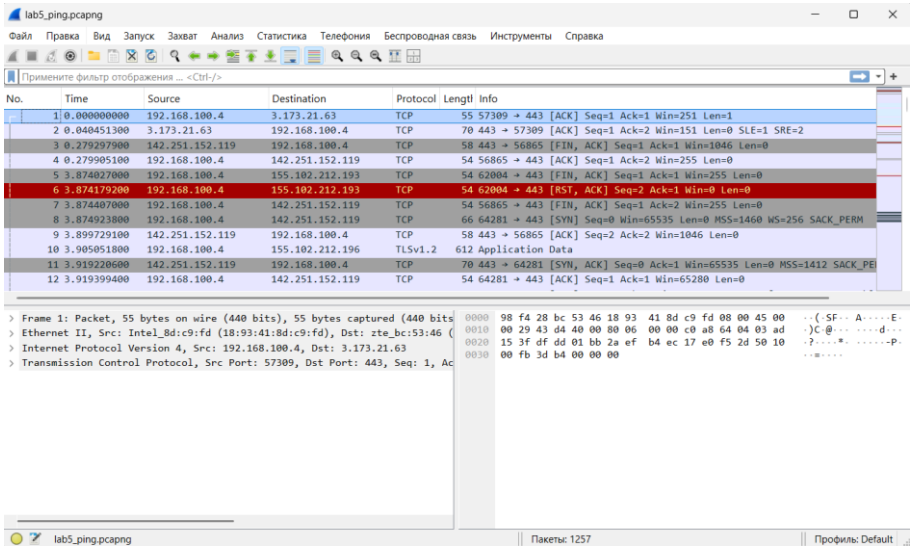
```
C:\Users\MSI>ping 192.168.100.1

Обмен пакетами с 192.168.100.1 по с 32 байтами данных:
Ответ от 192.168.100.1: число байт=32 время=3мс TTL=64
Ответ от 192.168.100.1: число байт=32 время=2мс TTL=64
Ответ от 192.168.100.1: число байт=32 время=2мс TTL=64
Ответ от 192.168.100.1: число байт=32 время=3мс TTL=64

Статистика Ping для 192.168.100.1:
    Пакетов: отправлено = 4, получено = 4, потеряно = 0
    (0% потерь)
Приблизительное время приема-передачи в мс:
    Минимальное = 2мсек, Максимальное = 3 мсек, Среднее = 2 мсек

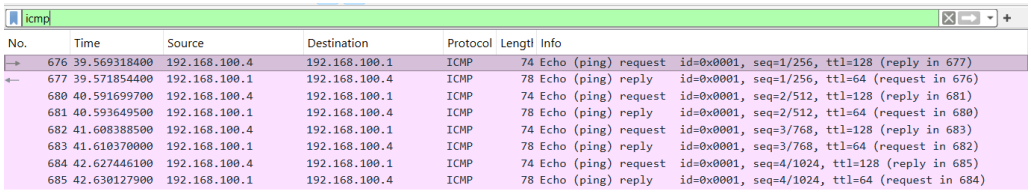
C:\Users\MSI>
```

Команда ping отправила 4 запроса и получила 4 ответа (0% потерь). Время отклика составило 2-3 мс. Значение TTL=64 указывает на то, что пакеты не проходили через промежуточные маршрутизаторы (шлюз находится в той же локальной сети).



Пакеты ICMP при выполнении команды ping (фильтр icmp):

С помощью фильтра отображения icmp были выделены пакеты, относящиеся к утилите. Утилита ping использует протокол ICMP, который работает поверх протокола IP (сетевой уровень), а тот инкапсулируется в кадры Ethernet. Перед отправкой первого ICMP-пакета использовался протокол ARP для определения MAC-адреса шлюза.



Структура заголовка ICMP (Echo request):

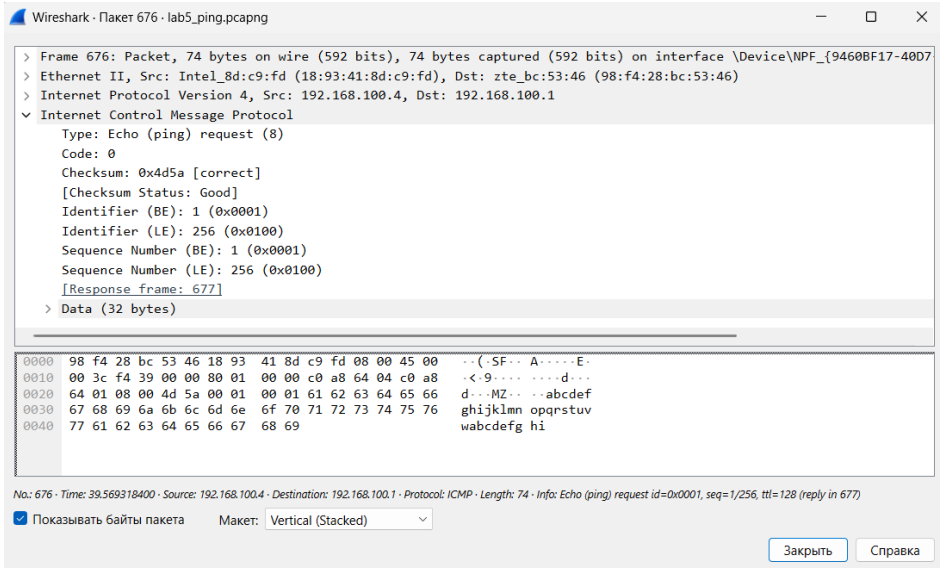
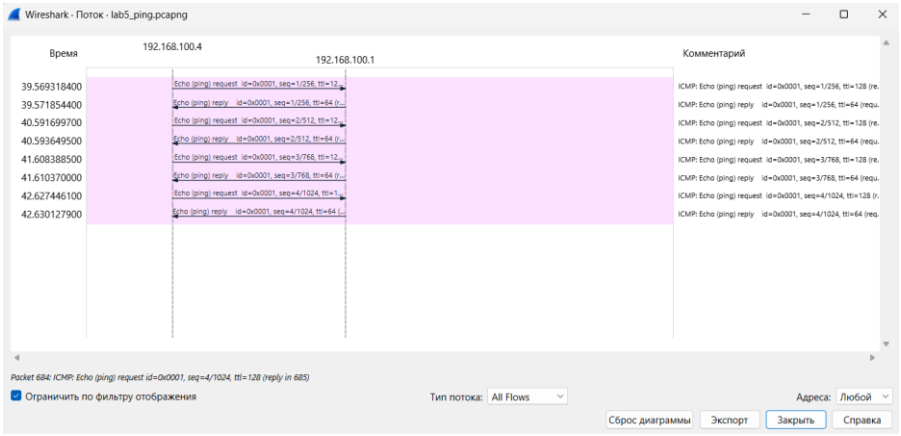


Диаграмма взаимодействия узлов при работе утилиты ping:



6. (TCP-сервер и TCP-клиент)

Компиляция:

```
g++ tcp_server.cpp -o tcp_server.exe -lws2_32
g++ tcp_client.cpp -o tcp_client.exe -lws2_32
```

Код программы:

tcp_server.cpp

```
#include <stdio.h>
#include <string.h>
#include <winsock2.h>
#include <windows.h>

#define MY_PORT 666

#define PRINTNUSERS if (nclients)\
    printf("%d user on-line\n",nclients);\
    else printf("No User on line\n");

DWORD WINAPI WorkWithClient(LPVOID client_socket);

int nclients = 0;

int main(int argc, char* argv[])
{
    char buff[1024];

    printf("SERVER\n\n");

    if (WSAStartup(0x0202,(WSADATA *) &buff[0]))
    {
        printf("Error WSAStartup %d\n", WSAGetLastError());
        return -1;
    }

    SOCKET mysocket;
    if ((mysocket=socket(AF_INET,SOCK_STREAM,0))==INVALID_SOCKET)
    {
        printf("Error socket %d\n",WSAGetLastError());
        WSACleanup();
```

```
        return -1;
    }

    sockaddr_in local_addr;
    local_addr.sin_family=AF_INET;
    local_addr.sin_port=htons(MY_PORT);
    local_addr.sin_addr.s_addr=0;

    if (bind(mysocket,(sockaddr *) &local_addr, sizeof(local_addr)))
    {
        printf("Error bind %d\n",WSAGetLastError());
        closesocket(mysocket);
        WSACleanup();
        return -1;
    }

    if (listen(mysocket, 0x100))
    {
        printf("Error listen %d\n",WSAGetLastError());
        closesocket(mysocket);
        WSACleanup();
        return -1;
    }

    printf("Waiting for connections, port %d\n", MY_PORT);

    SOCKET client_socket;
    sockaddr_in client_addr;
    int client_addr_size=sizeof(client_addr);

    while((client_socket=accept(mysocket, (sockaddr *)
            &client_addr, &client_addr_size))!=INVALID_SOCKET)
    {
        nclients++;

        HOSTENT *hst;
        hst=gethostbyaddr((char *)&client_addr.sin_addr.s_addr,4, AF_INET);

        printf("+%s [%s] new connect!\n",
            (hst)?hst->h_name:"",
            inet_ntoa(client_addr.sin_addr));
        PRINTNUSERS

        DWORD thID;
        CreateThread(NULL, 0, WorkWithClient,
            (LPVOID)client_socket, 0, &thID);
    }
    return 0;
}

DWORD WINAPI WorkWithClient(LPVOID client_socket)
{
    SOCKET my_sock;
    my_sock=(SOCKET)client_socket;
    char buff[20*1024];
    #define sHELLO "Hello, Student!\r\n"
```

```
    send(my_sock,sHELLO,strlen(sHELLO),0);

    char group[64];
    int filled = 0;
    int bytes_recv;

    while( (bytes_recv=recv(my_sock,&buff[0],sizeof(buff),0))
           && bytes_recv !=SOCKET_ERROR)
    {
        int i;
        for (i = 0; i < bytes_recv; i++)
        {
            group[filled++] = buff[i];
            if (filled < 64)
                continue;

            int counts[256];
            int order[256];
            int nunique = 0;
            int k;
            memset(counts, 0, sizeof(counts));
            for (k = 0; k < 64; k++)
            {
                unsigned char ch = (unsigned char)group[k];
                if (counts[ch] == 0)
                    order[nunique++] = ch;
                counts[ch]++;
            }

            char outmsg[1024];
            if (nunique < 3)
            {
                strcpy(outmsg,
                    "Less than 3 distinct characters. Connection
closed.\r\n");
                send(my_sock, outmsg, strlen(outmsg), 0);
                nclients--;
                printf("-disconnect (less than 3 distinct characters)\n");
                PRINTNUSERS
                closesocket(my_sock);
                return 0;
            }

            strcpy(outmsg, "<");
            for (k = 0; k < nunique; k++)
            {
                char piece[64];
                unsigned char ch = (unsigned char)order[k];
                char shown[8];
                if (ch == '\n') strcpy(shown, "\\n");
                else if (ch == '\r') strcpy(shown, "\\r");
                else if (ch == '\t') strcpy(shown, "\\t");
                else { shown[0] = (char)ch; shown[1] = 0; }
                sprintf(piece, "%s'%s-%d'",
                    k ? ", " : "",
```

```
                    shown, counts[ch]);
                strcat(outmsg, piece);
            }
            strcat(outmsg, ">\r\n");
            send(my_sock, outmsg, strlen(outmsg), 0);
            printf("group processed: %s", outmsg);
            fflush(stdout);
            filled = 0;
        }
    }

    nclients--;
    printf("-disconnect\n");
    PRINTNUSERS
    closesocket(my_sock);
    return 0;
}
```

tcp_client.cpp

```
#include <stdio.h>
#include <string.h>
#include <time.h>
#include <conio.h>
#include <winsock2.h>
#include <windows.h>

#define PORT 666
#define SERVERADDR "127.0.0.1"

SOCKET g_sock = INVALID_SOCKET;
FILE *g_log = NULL;
int g_connected = 0;

void log_event(const char *what, const char *detail)
{
    time_t t;
    struct tm *tm;
    char ts[64];
    if (!g_log)
        return;
    time(&t);
    tm = localtime(&t);
    sprintf(ts, "%04d-%02d-%02d %02d:%02d:%02d",
              tm->tm_year + 1900, tm->tm_mon + 1, tm->tm_mday,
              tm->tm_hour, tm->tm_min, tm->tm_sec);
    fprintf(g_log, "%s %s %s\n", ts, what, detail ? detail : "");
    fflush(g_log);
}

DWORD WINAPI RecvThread(LPVOID)
{
    char buff[4096];
    int n;
    while ((n = recv(g_sock, &buff[0], sizeof(buff) - 1, 0)) > 0)
```

```
    {
        buff[n] = 0;
        printf("S=>C:%s", buff);
        if (n > 0 && buff[n - 1] != '\n')
            printf("\n");
        log_event("RECEIVED", buff);
    }
    printf("\nConnection closed by server\n");
    log_event("CONNECTION END", "");
    g_connected = 0;
    return 0;
}

int main(int argc, char *argv[])
{
    char buff[1024];

    printf("CLIENT\n\n");
    printf("Auto-connect to %s:%d\n", SERVERADDR, PORT);
    printf("Type text, send - PgDn or Enter, exit - quit/exit + PgDn or
Enter\n\n");

    g_log = fopen("tcp-client.log", "a");

    if (WSAStartup(0x202, (WSADATA *)&buff[0]))
    {
        printf("WSAStart error %d\n", WSAGetLastError());
        return -1;
    }

    g_sock = socket(AF_INET, SOCK_STREAM, 0);
    if (g_sock == INVALID_SOCKET)
    {
        printf("Socket() error %d\n", WSAGetLastError());
        return -1;
    }

    sockaddr_in dest_addr;
    dest_addr.sin_family = AF_INET;
    dest_addr.sin_port = htons(PORT);
    HOSTENT *hst;

    if (inet_addr(SERVERADDR) != INADDR_NONE)
        dest_addr.sin_addr.s_addr = inet_addr(SERVERADDR);
    else if (hst = gethostbyname(SERVERADDR))
        ((unsigned long *)&dest_addr.sin_addr)[0] =
             ((unsigned long **)hst->h_addr_list)[0][0];
    else
    {
        printf("Invalid address %s\n", SERVERADDR);
        closesocket(g_sock);
        WSACleanup();
        return -1;
    }

    if (connect(g_sock, (sockaddr *)&dest_addr, sizeof(dest_addr)))
```

```
{
    printf("Connect error %d\n", WSAGetLastError());
    return -1;
}

printf("Connection to %s established successfully\n", SERVERADDR);
log_event("CONNECTION START", SERVERADDR ":666");
g_connected = 1;

DWORD thID;
CreateThread(NULL, 0, RecvThread, NULL, 0, &thID);

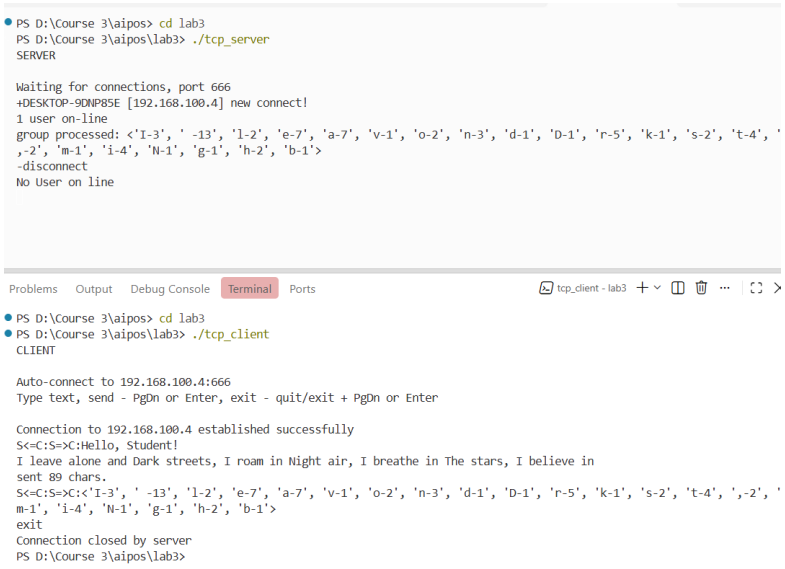
char line[1024];
int len = 0;
printf("S<=C:");
fflush(stdout);

while (g_connected)
{
    int c = getch();
    if (c == 27)
        break;
    if (c == 0 || c == 224)
    {
        int k = getch();
        if (k == 81 && len > 0)   /* PgDn */
        {
            line[len] = 0;
            if (!strcmp(line, "quit") || !strcmp(line, "exit"))
                break;
            send(g_sock, line, len, 0);
            printf("\nsent %d chars.\nS<=C:", len);
            log_event("SENT", line);
            len = 0;
        }
        continue;
    }
    if (c == '\r' || c == '\n')
    {
        if (len > 0)
        {
            line[len] = 0;
            if (!strcmp(line, "quit") || !strcmp(line, "exit"))
                break;
            send(g_sock, line, len, 0);
            printf("\nsent %d chars.\nS<=C:", len);
            log_event("SENT", line);
            len = 0;
        }
        continue;
    }
    if (c == 8)
    {
        if (len > 0)
        {
            len--;
```

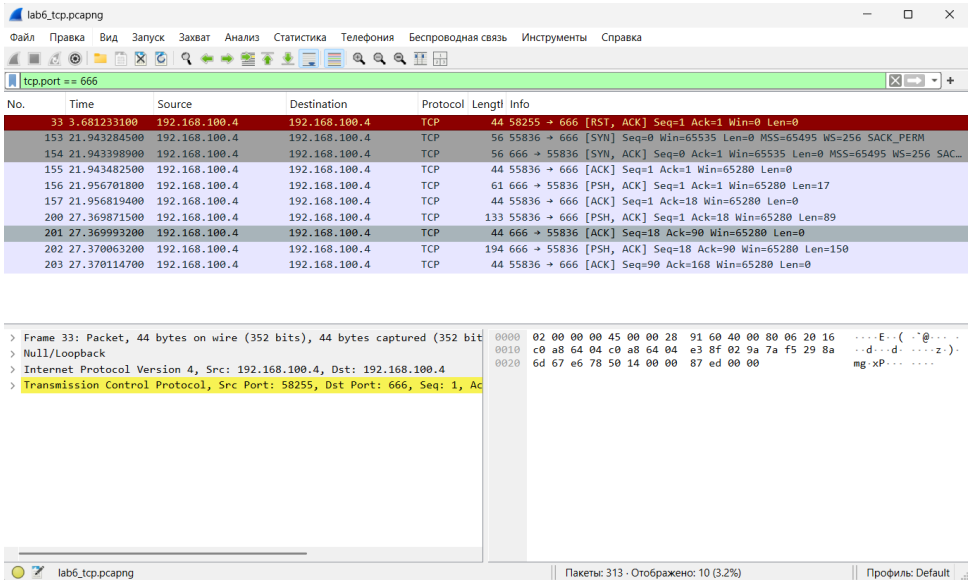
```
                printf("\b \b");
                fflush(stdout);
            }
            continue;
        }
        if (c >= 32 && len < (int)sizeof(line) - 1)
        {
            line[len++] = (char)c;
            putchar(c);
            fflush(stdout);
        }
    }

    log_event("CONNECTION END", "");
    closesocket(g_sock);
    WSACleanup();
    if (g_log)
        fclose(g_log);
    return 0;
}
```

Результат программы:



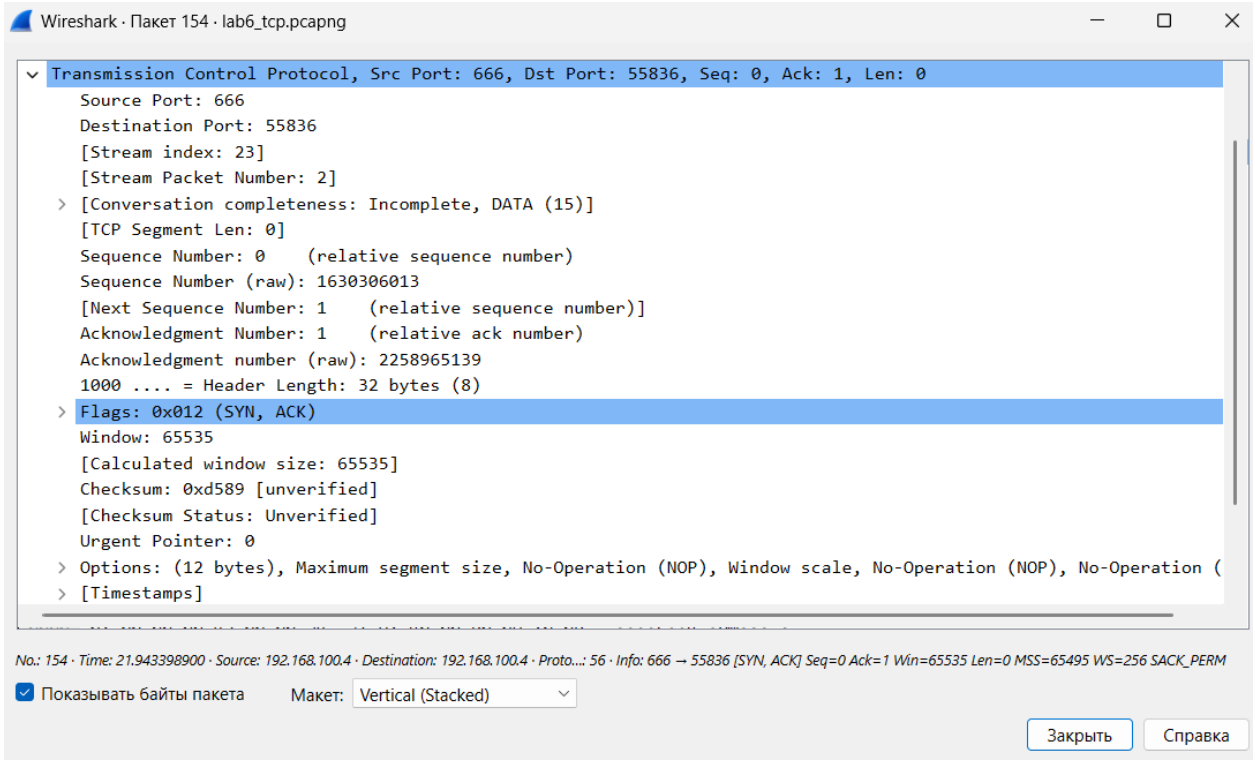
Для анализа протокола TCP был запущен захват трафика на интерфейсе Adapter for loopback traffic capture (так как сервер и клиент запускались на одной машине). Затем был запущен разработанный TCP-сервер и TCP-клиент. В качестве фильтра отображения использовался tcp.port == 666.



Видно, как программа установила соединение: сначала **SYN** (запрос на подключение), потом **SYN, ACK** (сервер согласился), и **ACK**. Потом пошла передача данных (строки PSH, ACK с длиной Len=17, Len=89 и т.д.).

Четко прослеживается установление TCP-соединения (трехэтапное рукопожатие):

1. Клиент отправляет пакет с флагом **SYN**;
2. Сервер отвечает пакетом с флагами **SYN, ACK**;
3. Клиент подтверждает установку соединения пакетом **ACK**.



Структура TCP-сегмента:

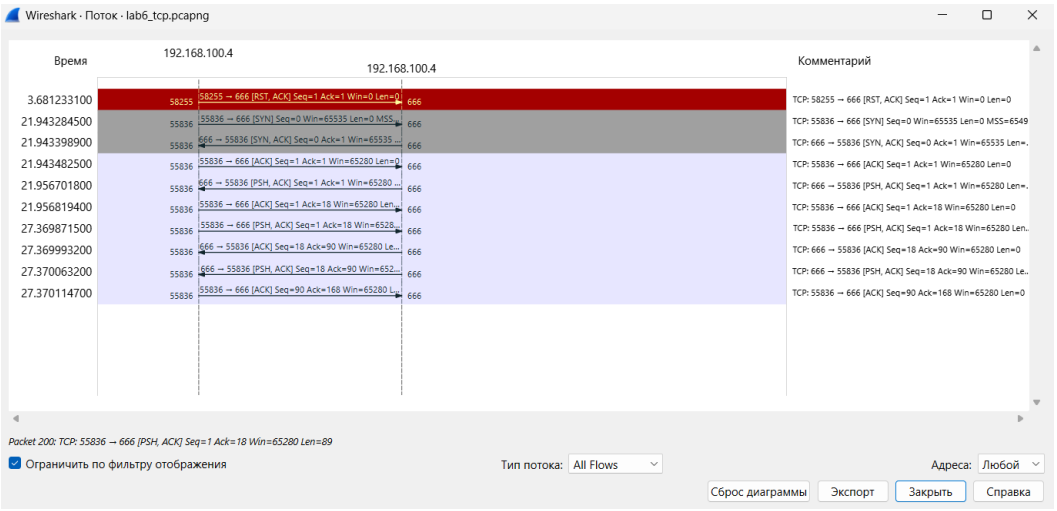
На примере пакета №154 (SYN, ACK) были выделены следующие поля заголовка TCP:

- Source Port (Порт отправителя): 666 (порт, на котором слушает сервер);
- Destination Port (Порт получателя): 55836 (динамический порт клиента);
- Sequence Number (Порядковый номер): 0 (относительный) / 1630306013 (сырой);

- Acknowledgment Number (Номер подтверждения): 1 (подтверждает получение SYN от клиента);
- Header Length (Длина заголовка): 32 байта (8) (20 байт базовая часть + 12 байт опций);
- Flags (Флаги): 0x012 (SYN, ACK) — подтверждает установку соединения;
- Window Size (Размер окна): 65535 — размер буфера для приема данных без подтверждения;
- Checksum (Контрольная сумма): 0xd589;
- Urgent Pointer (Указатель срочности): 0.

В поле **Options (Опции)** видны параметры Maximum segment size (максимальный размер сегмента) и Window scale (масштабирование окна), которые стороны согласовывают при подключении.

Диаграмма взаимодействия TCP-сервера и TCP-клиента:



7. UDP-сервер и UDP-клиент

Компиляция:

```
g++ udp_server.cpp -o udp_server.exe -lws2_32
g++ udp_client.cpp -o udp_client.exe -lws2_32
```

Код программы:

udp_server.cpp

```
#define _WINSOCK_DEPRECATED_NO_WARNINGS

#include <stdio.h>
#include <string.h>
#include <winsock2.h>
#include <windows.h>
#include <unordered_map>
#include <string>

#pragma comment(lib, "ws2_32.lib")

#define MY_PORT 666
#define GROUP_SIZE 64
#define BUFFER_SIZE 20480
```

```
struct ClientState
{
    char group[GROUP_SIZE];
    int filled;
    int stopped;
};

std::unordered_map<std::string, ClientState> clients;

std::string client_key(const sockaddr_in& addr)
{
    return std::string(inet_ntoa(addr.sin_addr)) + ":" +
           std::to_string(ntohs(addr.sin_port));
}

void send_message(SOCKET sock, const sockaddr_in& addr,
                  const char* message)
{
    sendto(sock, message, (int)strlen(message), 0,
           (const sockaddr*)&addr, sizeof(addr));
}

void process_group(SOCKET sock, const sockaddr_in& addr,
                   ClientState& state)
{
    int counts[256] = {};
    int order[256];
    int unique_count = 0;

    for (int i = 0; i < GROUP_SIZE; i++)
    {
        unsigned char ch = (unsigned char)state.group[i];

        if (counts[ch] == 0)
            order[unique_count++] = ch;

        counts[ch]++;
    }

    char outmsg[4096];

    if (unique_count < 3)
    {
        strcpy_s(outmsg, sizeof(outmsg),
            "ERROR: Less than 3 distinct characters. Session ended.");

        send_message(sock, addr, outmsg);

        state.filled = 0;
        state.stopped = 1;

        printf("%s\n", outmsg);
        return;
    }

    strcpy_s(outmsg, sizeof(outmsg), "<");
```

```
    for (int i = 0; i < unique_count; i++)
    {
        unsigned char ch = (unsigned char)order[i];
        char piece[64];

        if (ch >= 32 && ch <= 126 && ch != '\'' && ch != '\\')
        {
            sprintf_s(piece, sizeof(piece),
                "%s'%c-%d'",
                i ? ", " : "",
                ch,
                counts[ch]);
        }
        else
        {
            sprintf_s(piece, sizeof(piece),
                "%s0x%02X-%d",
                i ? ", " : "",
                ch,
                counts[ch]);
        }

        strcat_s(outmsg, sizeof(outmsg), piece);
    }

    strcat_s(outmsg, sizeof(outmsg), ">");

    send_message(sock, addr, outmsg);

    printf("Group processed: %s\n", outmsg);

    state.filled = 0;
}

int main()
{
    printf("UDP SERVER\n\n");

    WSADATA wsaData;

    if (WSAStartup(MAKEWORD(2, 2), &wsaData))
    {
        printf("WSAStartup error %d\n", WSAGetLastError());
        return 1;
    }

    SOCKET mysocket = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);

    if (mysocket == INVALID_SOCKET)
    {
        printf("Socket error %d\n", WSAGetLastError());
        WSACleanup();
        return 1;
    }
```

```
sockaddr_in local_addr = {};

local_addr.sin_family = AF_INET;
local_addr.sin_port = htons(MY_PORT);
local_addr.sin_addr.s_addr = htonl(INADDR_ANY);

if (bind(mysocket, (sockaddr*)&local_addr,
         sizeof(local_addr)) == SOCKET_ERROR)
{
    printf("Bind error %d\n", WSAGetLastError());

    closesocket(mysocket);
    WSACleanup();

    return 1;
}

printf("Waiting for UDP datagrams, port %d\n", MY_PORT);

char buff[BUFFER_SIZE];

while (true)
{
    sockaddr_in client_addr = {};
    int client_addr_size = sizeof(client_addr);

    int bytes_recv = recvfrom(
        mysocket,
        buff,
        sizeof(buff),
        0,
        (sockaddr*)&client_addr,
        &client_addr_size
    );

    if (bytes_recv == SOCKET_ERROR)
    {
        printf("recvfrom error %d\n", WSAGetLastError());
        continue;
    }

    std::string key = client_key(client_addr);

    printf("Received from %s: ", key.c_str());

    fwrite(buff, 1, bytes_recv, stdout);
    printf("\n");

    std::string message(buff, bytes_recv);

    if (message == "QUIT")
    {
        clients.erase(key);

        send_message(mysocket, client_addr, "Session ended.");
```

```
            printf("Session reset for %s\n", key.c_str());
            continue;
        }

        ClientState& state = clients[key];

        if (state.stopped)
            continue;

        for (int i = 0; i < bytes_recv; i++)
        {
            state.group[state.filled++] = buff[i];

            if (state.filled == GROUP_SIZE)
            {
                process_group(mysocket, client_addr, state);

                if (state.stopped)
                    break;
            }
        }
    }

    closesocket(mysocket);
    WSACleanup();

    return 0;
}
```

udp_client.cpp

```
#define _WINSOCK_DEPRECATED_NO_WARNINGS

#include <stdio.h>
#include <string.h>
#include <time.h>
#include <conio.h>
#include <winsock2.h>
#include <windows.h>

#pragma comment(lib, "ws2_32.lib")

#define PORT 666
#define SERVERADDR "127.0.0.1"

SOCKET g_sock = INVALID_SOCKET;
FILE* g_log = NULL;
volatile LONG g_connected = 0;
CRITICAL_SECTION g_log_lock;

void log_event(const char* what, const char* detail)
{
    if (!g_log)
```

```
        return;

    EnterCriticalSection(&g_log_lock);

    time_t t;
    time(&t);

    struct tm tm_info;
    localtime_s(&tm_info, &t);

    char ts[64];

    sprintf_s(ts, sizeof(ts),
        "%04d-%02d-%02d %02d:%02d:%02d",
        tm_info.tm_year + 1900,
        tm_info.tm_mon + 1,
        tm_info.tm_mday,
        tm_info.tm_hour,
        tm_info.tm_min,
        tm_info.tm_sec);

    fprintf(g_log, "%s %s %s\n",
        ts, what, detail ? detail : "");

    fflush(g_log);

    LeaveCriticalSection(&g_log_lock);
}

DWORD WINAPI RecvThread(LPVOID)
{
    char buff[4096];

    while (InterlockedCompareExchange(&g_connected, 1, 1))
    {
        int n = recv(g_sock, buff, sizeof(buff) - 1, 0);

        if (n == SOCKET_ERROR)
        {
            if (InterlockedCompareExchange(&g_connected, 1, 1))
            {
                printf("\nReceive error: %d\n", WSAGetLastError());
                log_event("RECEIVE ERROR", "");
            }
            break;
        }

        if (n == 0)
            continue;

        buff[n] = 0;

        printf("\nS=>C: %s\n", buff);

        log_event("RECEIVED", buff);
```

```
        if (strncmp(buff, "ERROR:", 6) == 0)
        {
            InterlockedExchange(&g_connected, 0);
            break;
        }
    }

    return 0;
}

int main()
{
    printf("UDP CLIENT\n\n");
    printf("Auto-start with server %s:%d\n", SERVERADDR, PORT);
    printf("Type text and send with PgDn or Enter.\n");
    printf("Type quit or exit and press PgDn or Enter to stop.\n\n");

    InitializeCriticalSection(&g_log_lock);

    g_log = fopen("udp-client.log", "a");

    WSADATA wsaData;

    if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0)
    {
        printf("WSAStartup error %d\n", WSAGetLastError());

        if (g_log)
            fclose(g_log);

        DeleteCriticalSection(&g_log_lock);
        return 1;
    }

    g_sock = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);

    if (g_sock == INVALID_SOCKET)
    {
        printf("Socket error %d\n", WSAGetLastError());

        WSACleanup();

        if (g_log)
            fclose(g_log);

        DeleteCriticalSection(&g_log_lock);
        return 1;
    }

    sockaddr_in dest_addr = {};

    dest_addr.sin_family = AF_INET;
    dest_addr.sin_port = htons(PORT);
    dest_addr.sin_addr.s_addr = inet_addr(SERVERADDR);

    if (connect(g_sock, (sockaddr*)&dest_addr, sizeof(dest_addr))
```

```
    == SOCKET_ERROR)
{
    printf("Server address error %d\n", WSAGetLastError());

    closesocket(g_sock);
    WSACleanup();

    if (g_log)
        fclose(g_log);

    DeleteCriticalSection(&g_log_lock);
    return 1;
}

printf("UDP communication target: %s:%d\n",
    SERVERADDR, PORT);

log_event("CLIENT START", SERVERADDR ":666");

InterlockedExchange(&g_connected, 1);

HANDLE recvThread = CreateThread(
    NULL, 0, RecvThread, NULL, 0, NULL);

if (!recvThread)
{
    printf("CreateThread error %lu\n", GetLastError());

    InterlockedExchange(&g_connected, 0);
    closesocket(g_sock);
    WSACleanup();

    if (g_log)
        fclose(g_log);

    DeleteCriticalSection(&g_log_lock);
    return 1;
}

char line[1024];
int len = 0;

printf("S<=C: ");
fflush(stdout);

while (InterlockedCompareExchange(&g_connected, 1, 1))
{
    int c = _getch();

    if (c == 27)
        break;

    if (c == 0 || c == 224)
    {
        int k = _getch();
```

```
    if (k == 81 && len > 0)
    {
        line[len] = 0;

        if (!strcmp(line, "quit") || !strcmp(line, "exit"))
            break;

        int sent = send(g_sock, line, len, 0);

        if (sent == SOCKET_ERROR)
        {
            printf("\nSend error %d\n", WSAGetLastError());
            break;
        }

        log_event("SENT", line);

        printf("\nSent %d chars.\nS<=C: ", sent);

        len = 0;
        fflush(stdout);
    }

    continue;
}

if (c == '\r' || c == '\n')
{
    if (len > 0)
    {
        line[len] = 0;

        if (!strcmp(line, "quit") || !strcmp(line, "exit"))
            break;

        int sent = send(g_sock, line, len, 0);

        if (sent == SOCKET_ERROR)
        {
            printf("\nSend error %d\n", WSAGetLastError());
            break;
        }

        log_event("SENT", line);

        printf("\nSent %d chars.\nS<=C: ", sent);

        len = 0;
        fflush(stdout);
    }

    continue;
}

if (c == 8)
{
```

```
            if (len > 0)
            {
                len--;
                printf("\b \b");
                fflush(stdout);
            }

            continue;
        }

        if (c >= 32 && len < (int)sizeof(line) - 1)
        {
            line[len++] = (char)c;
            putchar(c);
            fflush(stdout);
        }
    }

    if (InterlockedCompareExchange(&g_connected, 1, 1))
    {
        const char* quitMessage = "QUIT";
        send(g_sock, quitMessage, (int)strlen(quitMessage), 0);
    }

    InterlockedExchange(&g_connected, 0);

    shutdown(g_sock, SD_BOTH);
    closesocket(g_sock);

    WaitForSingleObject(recvThread, 1000);
    CloseHandle(recvThread);

    log_event("CLIENT END", "");

    WSACleanup();

    if (g_log)
        fclose(g_log);

    DeleteCriticalSection(&g_log_lock);

    return 0;
}
```

Результат программы:

```
PS D:\Course 3\aipos> cd lab3
PS D:\Course 3\aipos\lab3> ./udp_server
UDP SERVER

Waiting for UDP datagrams, port 666
Received from 127.0.0.1:49667: I leave alone and Dark streets, I roam in Night air, I breathe in The stars, I believe
in
Group processed: <'I-3', ' -13', 'l-2', 'e-7', 'a-7', 'v-1', 'o-2', 'n-3', 'd-1', 'D-1', 'r-5', 'k-1', 's-2', 't-4', '
,-2', 'm-1', 'i-4', 'N-1', 'g-1', 'h-2', 'b-1'>
```

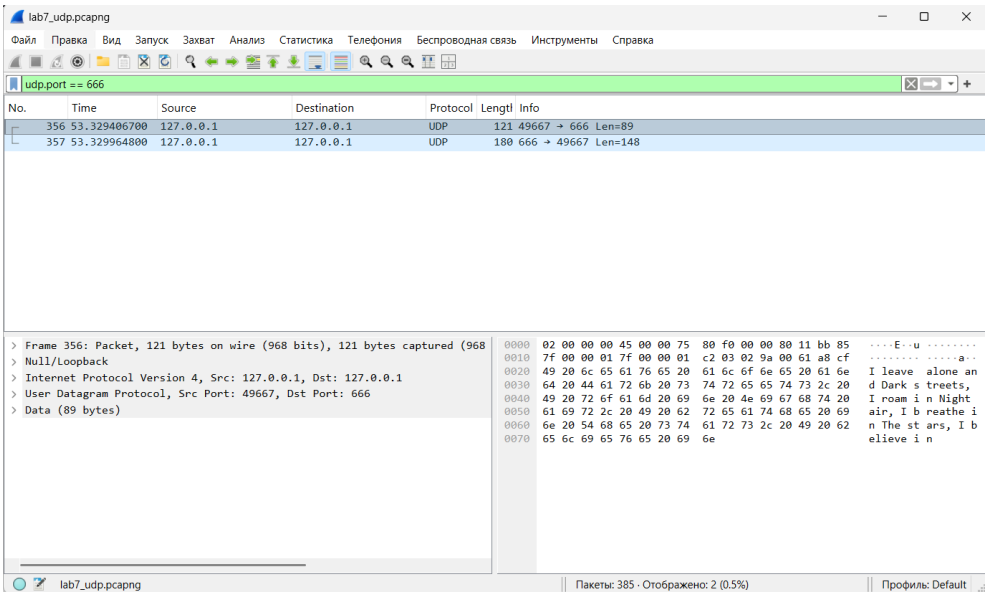
Problems Output Debug Console Terminal Ports udp_client - lab3

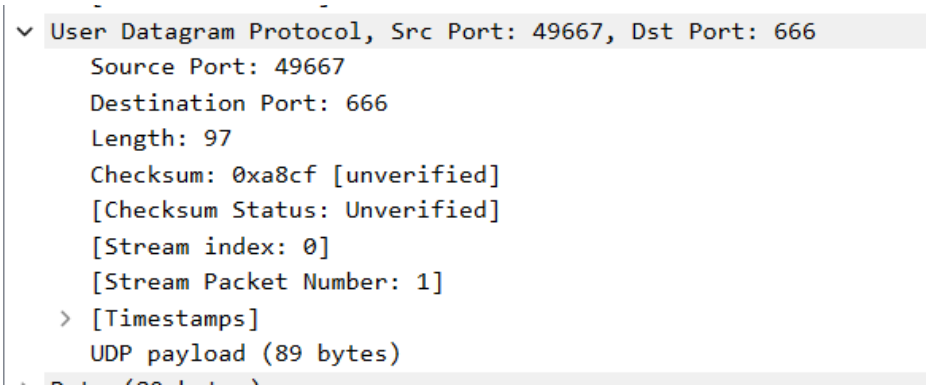
```
PS D:\Course 3\aipos> cd lab3
PS D:\Course 3\aipos\lab3> ./udp_client
UDP CLIENT

Auto-start with server 127.0.0.1:666
Type text and send with PgDn or Enter.
Type quit or exit and press PgDn or Enter to stop.

UDP communication target: 127.0.0.1:666
S<=C: I leave alone and Dark streets, I roam in Night air, I breathe in The stars, I believe in
Sent 89 chars.
S<=C:
S=>C: <'I-3', ' -13', 'l-2', 'e-7', 'a-7', 'v-1', 'o-2', 'n-3', 'd-1', 'D-1', 'r-5', 'k-1', 's-2', 't-4', ',-2', 'm-1'
, 'i-4', 'N-1', 'g-1', 'h-2', 'b-1'>
```

Для анализа протокола UDP был запущен захват трафика на интерфейсе Adapter for loopback traffic capture. Затем были запущены UDP-сервер и UDP-клиент. В качестве фильтра использовался udp.port == 666.
В отличие от TCP, протокол UDP не устанавливает соединение и не подтверждает доставку пакетов. На скриншоте видно, что клиент просто отправляет датаграмму на сервер, а сервер отвечает своей датаграммой. Никаких пакетов SYN или ACK здесь нет.



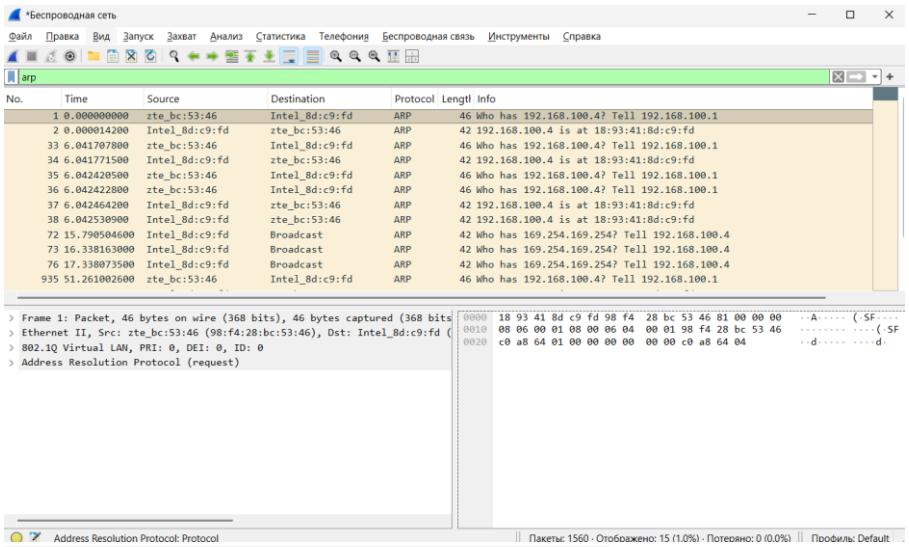


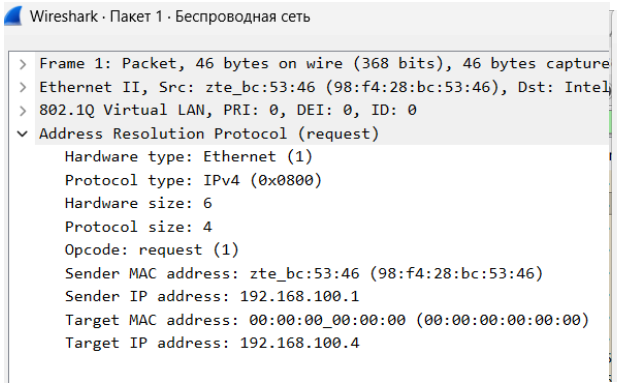
Структура UDP-сегмента:
На примере первого пакета (от клиента к серверу) были выделены следующие поля заголовка UDP (Рисунок 7.2):

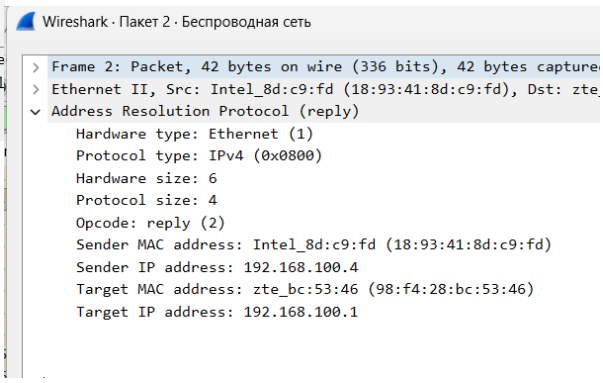
- Source Port (Порт отправителя): 49667 (динамический порт клиента);
- Destination Port (Порт получателя): 666 (порт, на котором слушает сервер);
- Length (Длина): 97 байт (8 байт заголовка UDP + 89 байт переданных данных);
- Checksum (Контрольная сумма): 0xa8cf.

8. Анализ ARP-протокола

Для исследования протокола ARP был запущен захват трафика на интерфейсе «Беспроводная сеть». С помощью команды ping был инициирован обмен данными с IP-адресом 192.168.100.1. В захвате были отфильтрованы пакеты с помощью фильтра arp. В результате были получены пакеты, демонстрирующие работу протокола ARP. В первом пакете шлюз 192.168.100.1 широковещательно запрашивает MAC-адрес нашей машины. Второй пакет является ответом от нашей машины, в котором она сообщает свой MAC-адрес.







Вывод: в ходе лабораторной работы были изучены возможности и приёмы работы со сниффером Wireshark, освоены фильтры захвата и фильтры отображения, а также приёмы статистической обработки трафика. Практически выполнен захват трафика при загрузке web-страницы, работе утилиты ping, TCP- и UDP-клиент-серверных приложений; для каждого случая получены статистические характеристики и проанализированы структуры заголовков Ethernet, IP, ICMP, TCP, UDP и ARP с интерпретацией значений полей; по результатам захватов построены диаграммы взаимодействия узлов.